DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS
College of Arts and Sciences | University of the Philippines Visayas
Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020
Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

Read, Convert, Store

CMSC 124 Activity 11

Prepared by: Rene Andre Bedonia Jocsing

Published: October 12, 2026 **Deadline:** October 14, 2026

A Java statement can choose an operator meaning, convert several values, mutate an array, yet skip half its work. Today you'll take those hidden steps apart and see each one.

This activity is written in Java. Java spells compound-assignment narrowing and constant-expression narrowing out as two separate rules. The third checkpoint sets them side by side. But in a language that folded the two together there would be nothing to set side by side. You aren't expected to know Java beyond what's on the page.

Laboratory Activity 3 asks the same questions of your own language: what a value's runtime type is, and what your evaluator does when the types don't line up. Activity 5 asked whether the two sides of an assignment already agreed, rejecting them when they didn't. Java converts instead. Today is about which conversion it picks, and when.

You have 60 minutes for the three checkpoints. Keep each worked example beside the checkpoint it supports. Discussion with classmates and the instructor is allowed and encouraged throughout. This activity is worth 16 points. Nine points reward traces, and seven reward explanations.

The Rules Java Applies

An **overloaded operator** has more than one meaning chosen from operand types. For `+`, two numeric operands request arithmetic, while a `String` operand requests concatenation. **Coercion** is an automatic type conversion used to make an operation legal.

A **widening conversion** moves to a type that can represent at least the source range, such as `byte` to `double`. A **narrowing conversion** moves to a smaller range and may lose information. Java's compound assignment performs the arithmetic, converts the result back to the target type, and evaluates the target location only once. **Numeric promotion** converts small integral operands such as `byte` to `int` before ordinary arithmetic. A **constant expression** is an expression the compiler can evaluate from constants alone. Java permits a fitting constant integer expression to narrow during assignment, but it doesn't grant the same conversion to a nonconstant arithmetic result.

Use these five rules:

1. `&&` evaluates its right operand only when its left operand is true. This is **short-circuit evaluation**.
2. Numeric addition with a `double` converts the other numeric operand to `double`.
3. Converting a positive `double` to `byte` discards the fractional part, then stores the low eight bits. Today's values stay within the byte range.
4. Method invocation widens a `byte` argument when the parameter type is `double`.
5. Java evaluates operands from left to right.

Read `&&` *as "conditional and,"* `index++` *as "use index, then increment it," and* `marks[index] += bonus` *as "select marks at index once, add bonus, narrow to the target type, and store." Read* `marks[index]` *as "marks at index" and* `note(value)` *as "call note with value."*

Build the Initializer One Piece at a Time

The initializer in the worked example after this block does six things in one line. Take them one at a time first. Begin with this state:

```
static int calls = 0;

static boolean note(double value) {
    calls++;
    return value > 6;
}

byte[] marks = {5};
int index = 0;
double bonus = 1.75;
```

Now start with a compound assignment on a fixed element:

```
marks[0] += bonus; // 5 at marks[0] widens to 5.0, plus 1.75 gives 6.75, narrows to byte 6
```

`marks[0]` now holds 6. The narrowing is the part that surprises people. `5.0 + 1.75` is a `double`. A plain `byte b = 5.0 + 1.75;` doesn't compile, but += carries its own conversion back to `byte`.

Now suppose the subscript moves while the target is being selected:

```
marks[index++] += bonus;    // the target is still marks[0]; index becomes 1 after lookup
```

Same result in the array, but `index` is now 1. The target was selected once, before the addition ran, so the increment ++ doesn't move where the store lands.

Now put a string beside the stored byte:

```
String piece = marks[0] + ":";    // piece = "6:"
```

That + chose concatenation because one operand is a `String`. The byte turned into the character `6`.

Now add the call on the end:

```
String whole = marks[0] + ":" + note(marks[0]);    // whole = "6:false"; calls is now 1
```

The second + concatenates as well, since its left operand, `marks[0] + ":"` which becomes `"6:"`, is already a `String`. `note` received byte 6 widened to double 6.0, bumped `calls` to 1, and returned false because 6.0 isn't greater than 6.

The worked example stacks those four pieces into one statement and puts a guard in front:

```
// Ren's Note: Do not ship code like this in production.
boolean ok = index < marks.length &&
    ((marks[index++] += bonus) + ":" + note(marks[index - 1]))
        .startsWith("7");
```

`index - 1` shows up because `index++` already moved `index` past the element that got written, so `marks[index - 1]` is the element the compound assignment stored into. `.startsWith("7")` asks one question about the finished string. Nothing new happens in the assembled version, except for the question and the guard. The bulk of the work all just occurs in one statement.

Worked Example for Checkpoint 1

Suggested time: 33 minutes, including the worked example.

```
static int calls = 0;

static boolean note(double value) {
    calls++;
    return value > 6;
}

byte[] marks = {5};
int index = 0;
double bonus = 1.75;

boolean ok = index < marks.length &&
    ((marks[index++] += bonus) + ":" + note(marks[index - 1]))
        .startsWith("7");
```

The guard `0 < 1` is true. `index++` selects `marks[0]`, then changes `index` to 1. The compound assignment saves that target, widens byte 5 to double 5.0, computes 6.75, narrows it to byte 6, and stores 6 in `marks[0]`.

The first operand after that assignment is byte 6, but the next operand is a string. The result becomes `"6:"`; the following + also concatenates. `note` receives byte 6 widened to double 6.0, increments `calls` to 1, and returns false. The string is `"6:false"`, so `startsWith("7")` is false. Final state: `marks = {6}`, `index = 1`, `calls = 1`, and `ok = false`.

The target-evaluated-once rule prevents a second increment. Using the new `index` during the store writes `marks[1]`, which is out of bounds, instead of the saved target `marks[0]`.

Compare two smaller conversion paths:

```
byte p = 5;
byte q = p + 1;  // does not compile: p + 1 has type int
p += 1.75;       // compiles: computes 6.75, then stores byte 6
```

Take note that ordinary assignment and compound assignment do not contain the same implicit conversion.

The four figures below are this initializer taken in order, from the guard through to the finished string:

```
boolean ok = index < marks.length &&
    ((marks[index++] += bonus) + ":" + note(marks[index - 1]))
        .startsWith("7");
```

One compound assignment hides several steps

The target is selected once, arithmetic happens at a wider type, and the result narrows before the later calls run.

Pass the guard

`index < marks.length` is true for `0 < 1`, so short-circuit evaluation proceeds to the right operand.

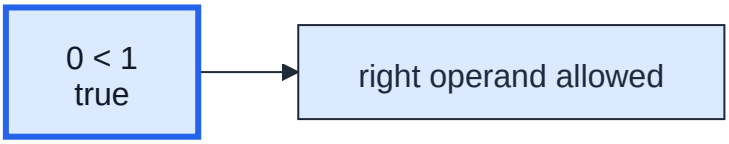


Figure 1: Pass the guard

Select the target once

`index++` selects `marks[0]` and then changes `index` to 1. The saved target remains element zero.

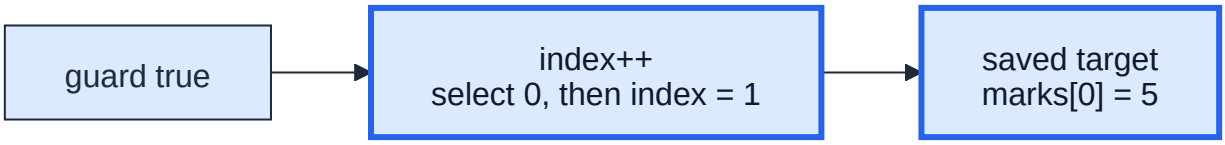


Figure 2: Select the target once

Widen, add, narrow, and store

The byte 5 widens to double, `5.0 + 1.75` produces 6.75, and compound assignment narrows that value to byte 6.

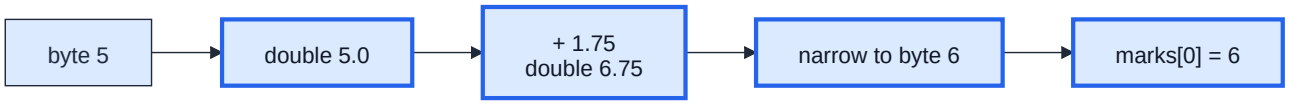


Figure 3: Widen, add, narrow, and store

Finish concatenation and the call

A string operand turns later + operations into concatenation. `note` receives 6.0, returns false, and the final string does not start with 7.

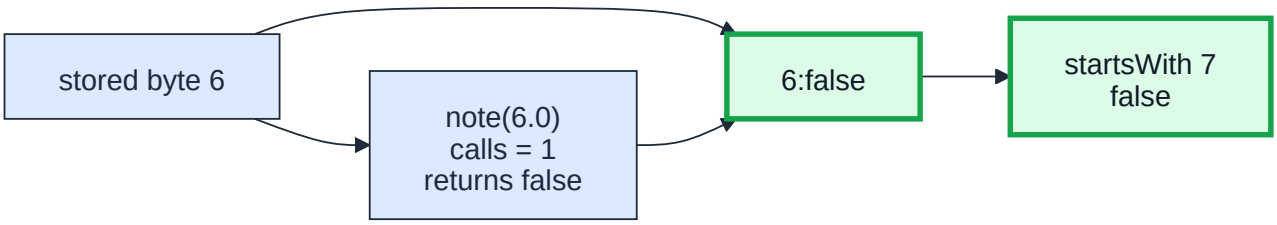


Figure 4: Finish concatenation and the call

Checkpoint 1: Expand the True Path (8 Points)

Running Case

```
static int calls = 0;

static boolean note(double value) {
    calls++;
```

```
    return value > 10;
}

byte[] levels = {4, 9};
int index = 1;
double bonus = 2.75;

boolean ok = index < levels.length &&
    ((levels[index++] += bonus) + ":" + note(levels[index - 1]))
        .startsWith("11");
```

`index++` produces the old value, then increments `index`. For the compound target, Java saves the array reference and old index before doing the addition. It doesn't evaluate `index++` again during the store.

For this initializer, evaluate operands left to right. A true `&&` guard permits the right operand. The compound assignment selects its array target once, widens the stored `byte` for addition with `double`, narrows the result back to `byte`, and stores it. Once + has a `String` operand, that operation and the following + concatenate.

1. (Construction: 5 points) Trace the initializer in evaluation order. Show the left side of `&&`, the saved array index, both numeric operand types, the arithmetic result before narrowing, the stored byte, each + selection, the argument received by `note`, and every final variable or array value.

2. (Explanation: 3 points) Answer all three, one point each. Cite the rule behind each answer.

a. Why does `index` increment exactly once?
b. Why does the fractional part disappear?
c. Why do the two + operations after the compound assignment select concatenation?

Transfer the Short Circuit

Worked Example for Checkpoint 2

Suggested time: 12 minutes, including the worked example.

For `int[] one = {4}; int i = 1;`, the left operand of `i < one.length && one[i] > 0` is false. Conditional `&&` skips `one[i] > 0`, so the array access, any call inside it, and any update inside it don't happen.

State after evaluation	Value
result	false
`i`	1
`one`	{4}

Evaluating both operands because they appear in the source ignores the operator's short-circuit behavior.

Checkpoint 2: Account for Skipped Work (4 Points)

Restart the complete running case with `levels = {4, 9}`, `index = 2`, `bonus = 2.75`, and `calls = 0`:

```
static int calls = 0;

static boolean note(double value) {
    calls++;
    return value > 10;
}
```

```

byte[] levels = {4, 9};
int index = 2; // Changed!
double bonus = 2.75;

boolean ok = index < levels.length &&
    ((levels[index++] += bonus) + ":" + note(levels[index - 1]))
        .startsWith("11");
```

Evaluate this initializer:

```
boolean ok = index < levels.length &&
    ((levels[index++] += bonus) + ":" + note(levels[index - 1]))
        .startsWith("11");
```

1. (Construction: 2 points) Trace the `&&` expression and record `ok`, `index`, `levels`, and `calls`. Mark the entire operand that isn't evaluated.

2. (Explanation: 2 points) Explain why the `index = 2` run skips the right operand entirely, leaving the array and `calls` unchanged.

Transfer the Conversion

Worked Example for Checkpoint 3

Suggested time: 15 minutes, including the worked example.

Compare these statements:

```
byte x = 8;
byte y = 8 + 1; // constant 9 fits, so this compiles
x += 1.5;       // computes 9.5, then stores byte 9
byte z = x + 1; // doesn't compile, because x isn't a constant and promotion occurs
```

The constant initializer for `y` receives the constant-expression exception. The compound assignment for `x` includes its own conversion back to `byte`. A wrong rule says every addition involving a byte returns a byte. Ordinary byte arithmetic is promoted to `int`, but a constant integer expression may be assigned to `byte` when its value fits. Compound assignment includes an implicit conversion back to its target type. The last statement fails because `x + 1` uses binary numeric promotion and evaluates into type `int`, becoming an expression that isn't assignable to byte.

Checkpoint 3: Decide Before Running (4 Points)

Three Fresh Fragments

Each fragment is compiled on its own, so no identifier in one collides with an identifier in another:

```
// A
byte a = 12;
byte b = a + 1;

// B
byte c = 12 + 1;

// C
byte d = 12;
d += 1.9;
```

1. (Construction: 2 points) For each fragment, state whether it compiles. For each compiling fragment, give the final byte value.

2. (Explanation: 2 points) Explain the different outcomes using promotion, constant-expression narrowing, and compound-assignment narrowing. Identify the wrong claim that all three additions follow the same conversion path.

End with the three decisions beside the conversion rule that governs each one.